mb-free___transport-rendezvous-128

An AgentMPI run analysis

Abstract

This is the floor of the rendezvous column in a ten-cell transport sweep — two modes crossed with five payload sizes, all at $p = 8$ and all agent-free. Eight ranks form a ring, each sending one 128-word payload to its successor under `Mode.RENDEZVOUS`, so only an envelope travels and the receiver would have to call `fetch` to see the text. It completed in 0.03 s over 34 events with zero admissions, a context high water of zero on every rank, and 1,344 tokens deferred. Its eager twin, at the same size, admitted 168 tokens into each receiver's context and also completed comfortably — so this cell is not evidence that rendezvous is necessary. It is the control that makes the cells above it interpretable, and it is the cleanest place in the sweep to establish what the deferred-token figure actually counts. The 1,344 is 8×168: one deferred send per rank, exactly equal to this run's `tokens sent`, because 100% of its traffic was rendezvous. The number recorded for this cell in `results/microbench/free-transport.json` is 2,688, twice that, and the discrepancy is worth chasing.

1 What this run is

property	value
run	`mb-free__transport-rendezvous-128`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	34
wall time	0.02 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	0 eager, 8 rendezvous
tokens sent	1,344	1,344 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

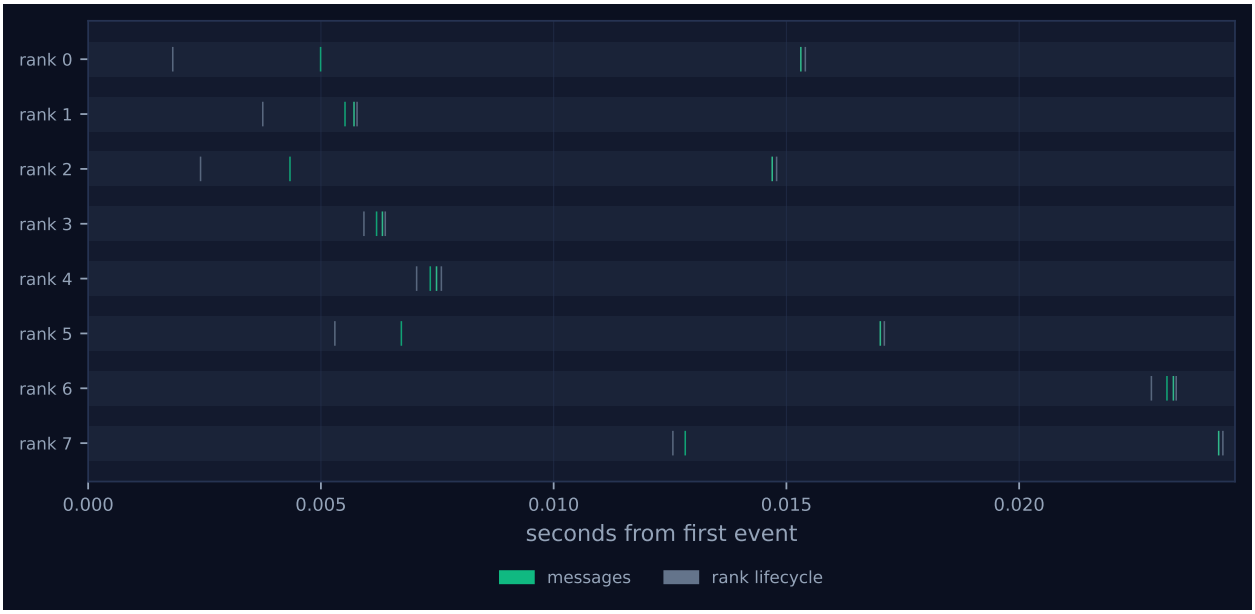

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

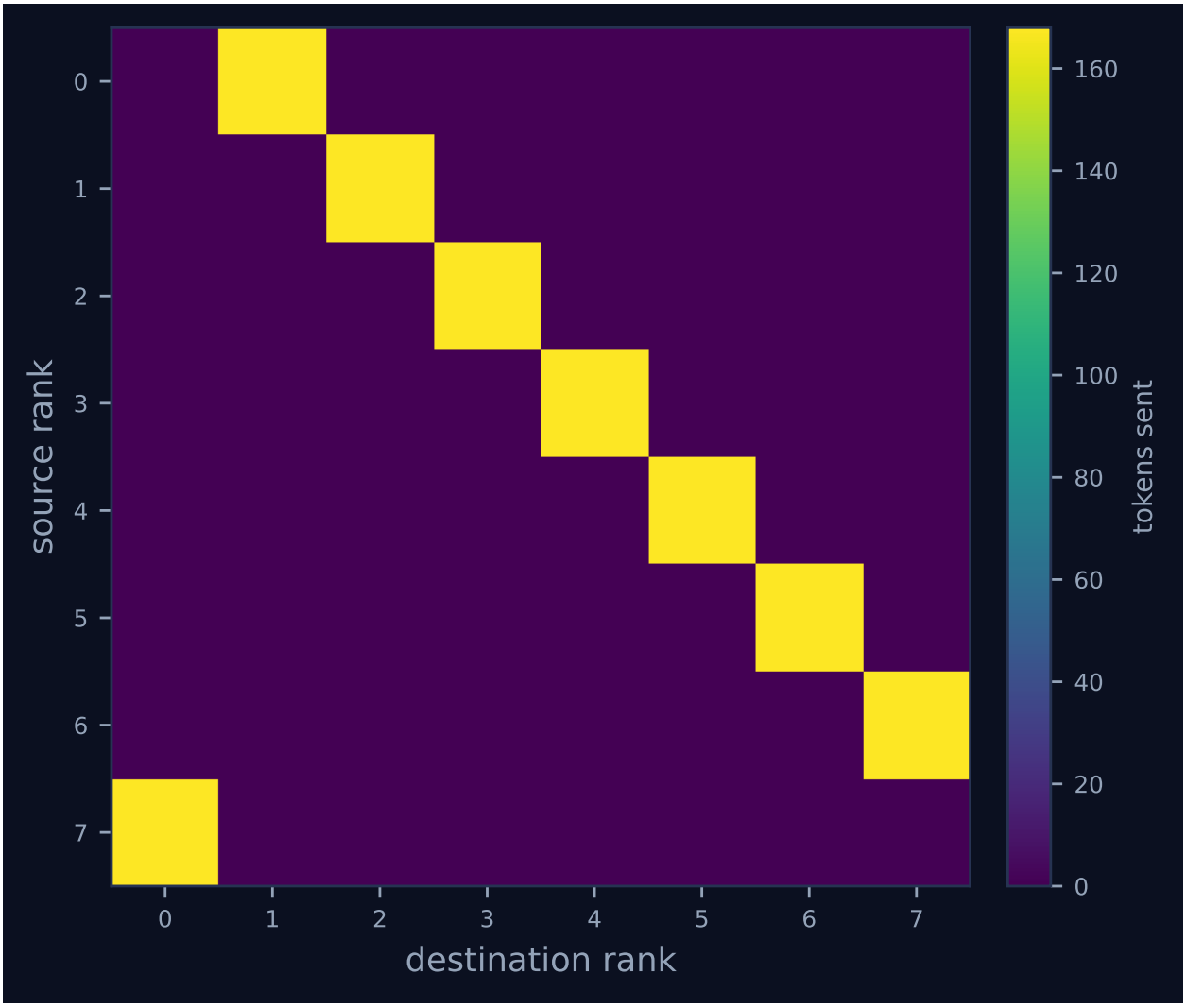

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	168	0.0	finalized
1	0.00	0.0	0	0	1	1	168	0.0	finalized
2	0.00	0.0	0	0	1	1	168	0.0	finalized
3	0.00	0.0	0	0	1	1	168	0.0	finalized
4	0.00	0.0	0	0	1	1	168	0.0	finalized
5	0.00	0.0	0	0	1	1	168	0.0	finalized
6	0.00	0.0	0	0	1	1	168	0.0	finalized
7	0.00	0.0	0	0	1	1	168	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes carrying two grey lifecycle diamonds and two green message ticks each, with no bars anywhere. The absence of extent is structural: the executor is `none` on all eight ranks, there were zero agent calls, and no `task.*` event exists for a bar to span, so the headline table's "idle fraction 100.0%" and "achieved parallelism 0.00×" record that no rank ever held a task rather than that ranks waited. Ranks arrive over a 21 ms window inside a 25 ms span — eight rank registrations against a single SQLite writer — and the ring's own work is the remainder. Median receive latency is 2.5 ms and the maximum 18 ms, and the maximum is not protocol delay: rank 6 initialised 23 ms in and claimed rank 5's message in the same millisecond, so its latency measures how long the message waited for a receiver that had not yet started. Figure 2 shows the topology the ticks cannot: one off-diagonal band, 168 tokens per edge from rank r to rank $(r{+}1)$ mod 8.

The row that separates this cell from its eager twin is in the rank health table: context occupancy 0% on all eight ranks, where `mb-free__transport-eager-128` reads 1%. Both runs wrote the same 640-byte blob to their content stores and both delivered eight envelopes; the difference is entirely in what `msg.recv` recorded. Every receive here carries `admitted: false` and `admitted_tokens: 0` against `tokens: 168`, and every `rank.finalize` reports `admissions: 0`, `n_items: 0`, `high_water: 0`. Nothing was read. The viewer's stats row makes the same point from the other side: `TOKENS DEFERRED 1.3k` where the eager cell shows `0.0k`.

7 Interpretation

Almost everything visible is by construction. The ring, the world size, the single send and receive per rank, and the mode are all harness choices, and the mode is forced twice over: `rank.init` records `eager_limit: 1000000000`, so `_choose_mode`'s automatic threshold is disabled, and a

168-token payload would in any case have gone eager under the shipped default of 2,048. The non-admission is also a default rather than a decision — the benchmark calls `recv` with no `admit` argument, and `Communicator._deliver` resolves `admit = None` to `mode != rendezvous`. That is the rule `comm.py` describes as the agent-world equivalent of MPI's convention that the sender's mode decides whether the receiver's unexpected-message pool is consumed, and it is why this run's zero occupancy required no receiver-side cooperation at all.

The deferred figure is where this cell earns its place. The log-derived total in `metrics.json` is 1,344, which is 8×168: eight ranks, one rendezvous send each, and no other traffic. It is exactly equal to `tokens sent`, as it must be when every message is rendezvous, and this identity is the sanity check the field is supposed to satisfy — deferral is a subset of what was sent. The results file records 2,688 for the same cell, and the per-rank `rank.finalize` events show where the extra comes from: each rank reports `tokens_deferred: 336` against `tokens_sent: 168`, a deferral larger than everything that rank transmitted. Until commit `0fea51d2`, `RankCost.tokens_deferred` accumulated on two axes at once, on the send side when transport was rendezvous and on the receive side whenever a message was not admitted, so each rank charged its own send once and its neighbour's arrival again. The fix separated the two into `tokens_deferred` and `tokens_unadmitted`, and under the current definitions this run would report 1,344 of each. Two consequences follow. The first is that the eager rows of the published transport table are unaffected and correctly read zero, because eager receives are admitted by the same default and neither counter fires — the observation this sweep's eager cells make. The second is that the rendezvous rows are affected after all, because this benchmark does *not* admit its rendezvous receives; nothing in it passes `admit` explicitly, and rendezvous defaults to false. Every rendezvous entry in that table is exactly twice the send-side deferral the field is documented to mean, at every one of the five sizes: 2,688, 10,784, 43,120, 172,464 and 689,856 against log-derived 1,344, 5,392, 21,560, 86,232 and 344,928. The trace viewer and `metrics.json` already show the corrected figures, because `cost.summarise` derives them from the log and was always right; only the recorded results carry the doubled number.

What this cell does not show is any benefit. At 168 tokens the eager twin used 0.5% of a receiver's budget and handed over the content; this run used none and handed over a handle nobody opened. Both completed, in 0.019 s and 0.025 s respectively, which at this scale is indistinguishable. Rendezvous is available here and it is not needed here, and saying so is the point of having the small cells: the sweep's argument is not that handles are always better but that the choice has to be expressible, and the cells that vindicate the choice are at 8,192 and 32,768 words, where the eager arm does not run at all. The generated finding list is empty and that is correct.

8 Threats to this reading

The zero-context result is real but cheap. This run never called `fetch`, so its receivers finished knowing a digest, a token count and a one-line synopsis and nothing else; the "saving" is the cost of not reading, and a harness that fetched with `admit=True` would land exactly where the eager twin did. The ping-pong benchmark in `experiments/microbench.py` shows the pattern this cell omits — `recv(admit=False)` followed by `comm.fetch(msg, admit=False)`, an explicit materialisation decision — and the transport sweep has no fetch anywhere. Second, the executor is `none` with zero agent calls, so every context figure here is an accounting entry in `ContextAccount` rather than a measured prompt, and nothing in the run bears on agent behaviour. Third, the token count is an estimate: provenance records `tokenizer_exact: false`, and 168 is $\lceil 640/3.8 \rfloor$ from a character heuristic applied to a payload of one repeated word, so both the deferred totals and the sweep's

ceiling shift by roughly 30% under a real tokeniser. Fourth, the wall time is one sample of 25 ms of which 21 ms is rank start-up against SQLite, so no timing comparison between this cell and its eager twin means anything. Finally, my reading of the doubled deferral rests on per-rank arithmetic — `tokens_deferred` 336 against `tokens_sent` 168, in a rank whose only actions were one rendezvous send and one non-admitted rendezvous receive of the same size — and on the pre-fix source. It is not an observation of the bug happening; the code that produced these traces is no longer in the tree.